

PROJECT ID: 36

S.NO	NAME	ROLL NO
1.	ANKIT KUMAR SHARMA	20252651007
2.	VIVEK KUMAR KOSTA	20252651065
3.	PIYUSH MENGHANI	20252651039
4.	VIJAY SINGH	20252651064
5.	SHUBHAM GUPTA	20252651021

1. Context

CampusEats integrates with **UniPay**, an external campus bank payment gateway, to charge student's cards. The integrated operation is ***charge***, which sends the card token, amount, and currency to UniPay and receives the payment result.

This edge uses Simple Object Access Protocol (SOAP) because the bank provides a strong, typed Web Services Description Language (WSDL) contract, message-level security through WS-Security, and standardized transaction and reliability features strictly required for financial operations. The rest of CampusEats will use REST because its own services are REST-shaped, while SOAP is used at the external enterprise edge where CampusEats integrates with a system it does not control.

- **Partner:** UniPay Bank Ltd.
- **Operation:** *charge*

2. HTTP Binding

The UniPay payment service utilizes SOAP over HTTP. The charge request is transmitted using an HTTP POST request to the endpoint URL obtained directly from the service/port binding definition within the WSDL.

SOAP/HTTP Request

```
POST /pay HTTP/1.1
Host: api.unipay.example
Content-Type: text/xml; charset=utf-8
SOAPAction: "urn:unipay:charge"

<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:pay="urn:unipay">

  <soap:Header>
    <pay:Credentials>
      <pay:merchantId>CAMPUS-EATS-001</pay:merchantId>
      <pay:apiKey>DEMO-API-KEY</pay:apiKey>
    </pay:Credentials>
  </soap:Header>

  <soap:Body>
    <pay:chargeRequest>
      <pay:cardToken>tok_9f2</pay:cardToken>
      <pay:amount>300.00</pay:amount>
      <pay:currency>INR</pay:currency>
    </pay:chargeRequest>
  </soap:Body>

</soap:Envelope>
```

Endpoint

Endpoint URL from service/port: <https://api.unipay.example/pay>

Binding Details

Property	Value
Method	POST
Host	api.unipay.example
Content-Type	text/xml; charset=utf-8
SOAPAction	"urn:unipay:charge"
Endpoint	https://api.unipay.example/pay

3. Discovery

CampusEats can obtain the UniPay WSDL document through two primary architectural patterns:

- 1. Direct URL Discovery:** UniPay provides CampusEats directly with a secure, known network URL to access its `Payments.wsdl` document.
- 2. Registry Lookup:** CampusEats queries an enterprise service catalogue or UDDI registry for a designated payment service, discovers UniPay, and retrieves its associated WSDL binding metadata.

Catalogue / Registry Entry

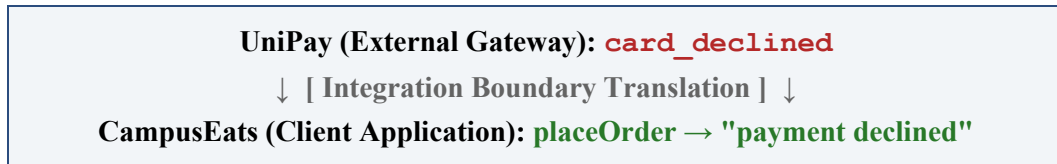
Field	Entry
Business	UniPay Bank Ltd.
Service	Payments
Endpoint	<code>https://api.unipay.example/pay</code>
WSDL / tModel	<code>urn:unipay:payments Payments WSDL</code>

The registry is strictly used to discover the enterprise partner and its WSDL contract. Following successful discovery, CampusEats directly binds to the UniPay endpoint URL and invokes the financial transaction service using SOAP over HTTP.

4. Fault Mapping

When UniPay rejects a card payment request, it generates and returns a formal SOAP Fault payload containing the partner-specific error code **card_declined**. CampusEats intercepts this message at the integration boundary and translates the partner error into its internal domain contract.

Error Mapping Flow



Integration Abstraction Principle

The error code **card_declined** is UniPay's partner-specific error definition. CampusEats abstracts and encapsulates this error at the system integration boundary. Consequently, the internal UniPay-specific error code and banking vocabulary are never exposed to students or client-facing application layers. CampusEats seamlessly handles the partner fault and presents its standardized **"payment declined"** exception message within the *placeOrder* workflow. This design strictly upholds the separation of concerns between external partner contracts and CampusEats' internal service contract architecture.